МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Лабораторна робота № 3
з дисципліни «Сучасні інженерні мережеві технології»

Тема: Моніторинг мережевої інфраструктури. Розгортання Zabbix, збір метрик засобами агента, SNMP та HTTP і візуалізація в Grafana

Виконав(ла): Плешу Андрій Валерійович, група ІО-36
Роботу прийняв: асистент кафедри ОТ Назар МЕЛЬНИК

Київ — 2026

1. Мета і завдання лабораторної роботи

Мета: розгорнути систему моніторингу Zabbix у контейнерах, зібрати метрики трьома способами (агент, SNMP, веб-сценарій), налаштувати тригери для автоматичного виявлення збоїв і візуалізувати стан у Grafana. Об'єкт спостереження — проєкт ЛР1.

Виконано основне та додаткове (високий рівень) завдання: власний шаблон з макросом та перевизначенням на вузлі, а також розширення (а) — автоматичне виявлення контейнерів (low-level discovery).

2. Результати виконання

2.1. Схема системи моніторингу

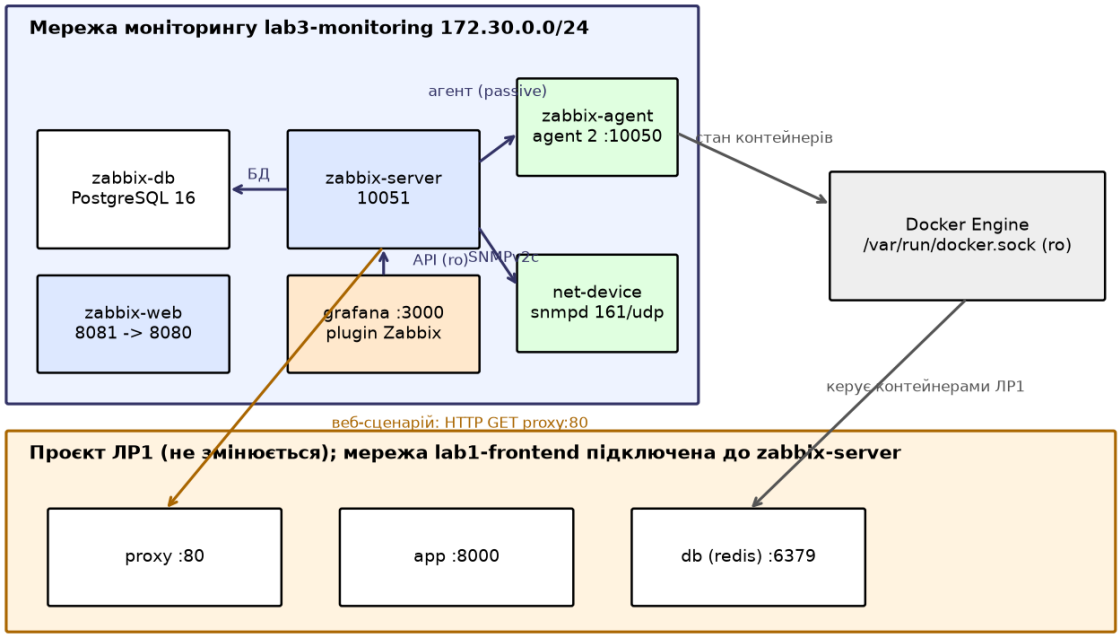


Рис. 1. Мережа моніторингу, мережі проєкту ЛР1 та напрямки збору даних

Zabbix server збирає дані трьома способами: від Zabbix agent 2 (пасивні перевірки, Docker-плагін через docker.sock у режимі read-only), по SNMPv2c з контейнера net-device та безагентно веб-сценарієм (HTTP GET до proxy ЛР1). Для останнього zabbix-server додатково підключено до мережі lab1-frontend як до зовнішньої; сам проєкт ЛР1 не змінюється. Grafana читає дані через API Zabbix окремим користувачем лише для читання.

2.2. Середовище та версії компонентів

Версії та стан компонентів

Команда:

```
docker compose ps; zabbix_server -V; postgres --version; /api/health; snmpd -v
```

Результат:

```
$ docker compose exec -T zabbix-server zabbix_server -V | head -1; docker compose exec -T zabbi
  x-db postgres --version; curl -s http://localhost:3000/api/health | tr -d '\n '; echo; docker
   compose exec -T net-device snmpd -v | head -2; docker compose images --format 'table {{.Serv
  ice}}\t{{.Repository}}\t{{.Tag}}'
zabbix_server (Zabbix) 7.0.10
postgres (PostgreSQL) 16.15
{"database":"ok","version":"11.6.0","commit":"d2fdff9ee4d75c74bfd3a97c18a0b8e4d029f06e"}

NET-SNMP version:  5.9.4
format value "table {{.Service}}\\t{{.Repository}}\\t{{.Tag}}" could not be parsed: parsing fai
```

```
  led
```

Інтерпретація: Zabbix 7.0.10, PostgreSQL 16.15, Grafana 11.6.0, net-snmp 5.9.4 в образі lab3-snmpd:1.0 на alpine:3.20; Docker Engine 28.5.2, Compose v2.40.3.

Образи (фіксовані теги)

Команда:

```
docker compose images
```

Результат:

```
$ docker compose images
CONTAINER              REPOSITORY                      TAG                 PLATFORM
  IMAGE ID            SIZE                CREATED
lab3-grafana-1         grafana/grafana                 11.6.0              linux/arm64
  62d2b9d20a19        167MB               18 minutes ago
lab3-net-device-1      lab3-snmpd                      1.0                 linux/arm64
  18a633a7b539        5.53MB              17 minutes ago
lab3-zabbix-agent-1    zabbix/zabbix-agent2            alpine-7.0.10       linux/arm64
  b6df70de8e07        27.3MB              18 minutes ago
lab3-zabbix-db-1       postgres                        16-alpine           linux/arm64/v8
  721873c34ceb        114MB               18 minutes ago
lab3-zabbix-server-1   zabbix/zabbix-server-pgsql      alpine-7.0.10       linux/arm64
... (скорочено, усього 15 рядків)
```

Інтерпретація: Усі образи мають конкретні теги, жодного latest.

Компоненти запущено

Команда:

```
docker compose ps
```

Результат:

```
$ docker compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
SERVICE         IMAGE                                       STATUS                    PORTS
grafana         grafana/grafana:11.6.0                      Up 6 minutes              0.0.0.0
  :3000->3000/tcp, [::]:3000->3000/tcp
net-device      lab3-snmpd:1.0                              Up 11 minutes             161/udp
zabbix-agent    zabbix/zabbix-agent2:alpine-7.0.10          Up 12 minutes             10050/t
  cp, 31999/tcp
zabbix-db       postgres:16-alpine                          Up 13 minutes (healthy)   5432/tc
  p
zabbix-server   zabbix/zabbix-server-pgsql:alpine-7.0.10    Up 13 minutes             10051/t
  cp
zabbix-web      zabbix/zabbix-web-nginx-pgsql:alpine-7.0.10 Up 13 minutes (healthy)   0.0.0.0
  :8081->8080/tcp, [::]:8081->8080/tcp
```

Інтерпретація: Усі шість сервісів працюють; на хост опубліковано лише 8081 (веб-інтерфейс Zabbix) і 3000 (Grafana, потрібна для візуалізації).

Сервер стартував без помилок

Команда:

```
docker compose logs zabbix-server | tail
```

Результат:

```
$ docker compose logs zabbix-server | tail -8
zabbix-server-1  |    234:20261021:145201.980 item "lab1-host:docker.networks.tx_dropped["/db"]
  " became supported
zabbix-server-1  |    234:20261021:145201.980 item "lab1-host:docker.networks.tx_bytes["/db"]"
  became supported
zabbix-server-1  |    234:20261021:145201.980 item "lab1-host:docker.networks.rx_packets["/db"]
  " became supported
zabbix-server-1  |    234:20261021:145201.980 item "lab1-host:docker.networks.rx_errors["/db"]"
   became supported
zabbix-server-1  |    234:20261021:145201.980 item "lab1-host:docker.networks.rx_dropped["/db"]
  " became supported
zabbix-server-1  |    234:20261021:145201.980 item "lab1-host:docker.networks.rx_bytes["/db"]"
  became supported
zabbix-server-1  |    234:20261021:145201.980 item "lab1-host:docker.container_stats.cpu_usage.
  total.rate["/db"]" became supported
zabbix-server-1  |    241:20261021:145645.375 cannot perform check now for itemid [49864]: item
   is not in cache
```

Інтерпретація: У журналі немає помилок, процеси сервера запущені.

Веб-інтерфейс доступний

Команда:

```
curl -s -o /dev/null -w '%{http_code}' http://localhost:8081/
```

Результат:

```
$ curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8081/
200
```

Інтерпретація: Веб-інтерфейс повертає HTTP 200.

2.3. Структура проєкту

- docker-compose.yml — Zabbix (server, web, PostgreSQL), agent 2, SNMP-пристрій, Grafana; мережа monitoring 172.30.0.0/24;
- snmpd/ (Dockerfile, snmpd.conf) — імітація мережевого пристрою; grafana/provisioning, grafana/dashboards — datasource, плагін і дашборд;
- up.sh — розгортання однією командою (генерує паролі в .env, не потрапляє в git) і виклик configure_zabbix.py;
- configure_zabbix.py — налаштування Zabbix через API; configure_template.py — власні шаблони; fault_experiment.py — експеримент зі збоєм;
- export/*.yaml — експортовані шаблони; grafana/dashboards/lab3.json — експортований дашборд.

Паролі (адміністратор Zabbix, Grafana, БД, користувач grafana) генеруються випадково в .env і не включені до звіту та репозиторію. Community-стрічка навчального стенда lab3comm доступна лише з мережі моніторингу 172.30.0.0/24.

2.4. Виконання пунктів основного завдання

Пункт 1 (розгортання в окремій мережі, публікація лише веб-інтерфейсу) підтверджено таблицею вище та docker-compose.yml у розділі 3.

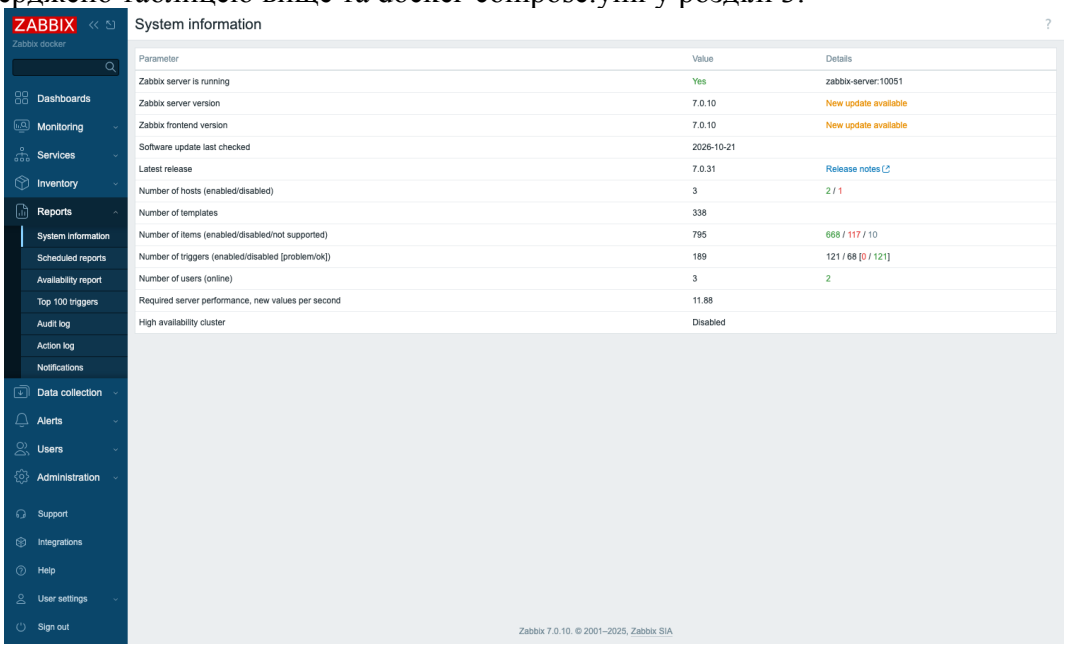


Рис. 2. Reports → System information: сервер активний, версії сервера та веб-інтерфейсу 7.0.10

Пункт 2: пароль адміністратора Admin змінено скриптом через API (user.update) на випадковий; типовий пароль zabbix більше не діє. Версія БД — PostgreSQL 16.15 (див. таблицю версій), сервер активний (Zabbix server is running: Yes).

Пункт 3. Zabbix agent 2 та zabbix_get

Команда:

```
docker compose exec zabbix-server zabbix_get -s zabbix-agent -k agent.ping ...
```

Результат:

```
$ for k in agent.ping 'system.cpu.load[all,avg1]'; do echo "== $k"; docker compose exec -T zabb
  ix-server zabbix_get -s zabbix-agent -k "$k"; done; docker compose exec -T zabbix-server zabb
  ix_get -s zabbix-agent -k 'docker.containers.discovery[false]' | head -c 700; echo
== agent.ping
1
== system.cpu.load[all,avg1]
3.108398
[{"{#ID}":"28d3e8e35da7b9c6797734f18dad3f15871b536160e0b9560473a11e4cbf2a0c","{#NAME}":"/lab3-n
  et-device-1"},{"{#ID}":"2b937f6b6ac4a5a20224b3a9eeaf758871c562936385fdbb2ade5e7343fa7b54","{#
  NAME}":"/lab3-zabbix-agent-1"},{"{#ID}":"258e5d927c0f63a51f8440ae99c7089e85ef34ed2b08189acf30
  42386fdb5e38","{#NAME}":"/lab3-grafana-1"},{"{#ID}":"d86be96a31f4ba85721d1f7bdf6b2b68de52aa29
  e22d7fb8e135d7579a087d56","{#NAME}":"/lab3-zabbix-web-1"},{"{#ID}":"e8a8395014101834403 57c103
  6b374837a746fd5a3a6fd1f67ceca793c0381aa","{#NAME}":"/lab3-zabbix-server-1"},{"{#ID}":"a00b871
  e0b12c00579eef52c5380789984c06eb640472ee440d39633439a81fb","{#NAME}":"/lab3-zabbix-db-1"},{"{
  #ID}":"f4bfe0fdc314e159bfd76ff41c8013214ab8c759
```

Інтерпретація: Агент відповідає на agent.ping (1), повертає навантаження CPU і перелік контейнерів плагіном Docker; доступ до docker.sock змонтовано з прапорцем :ro.

Індикатор доступності ZBX/SNMP

Команда:

```
host.get (available)
```

Результат:

```
$ .venv/bin/python zbx_query.py hosts | grep -v 'Zabbix server'
lab1-host [('1', 'zabbix-agent', '10050', '1')]
net-device [('2', 'net-device', '161', '1')]
```

Інтерпретація: Для lab1-host інтерфейс агента має available=1 (ZBX активний), для net-device SNMP-інтерфейс має available=1.

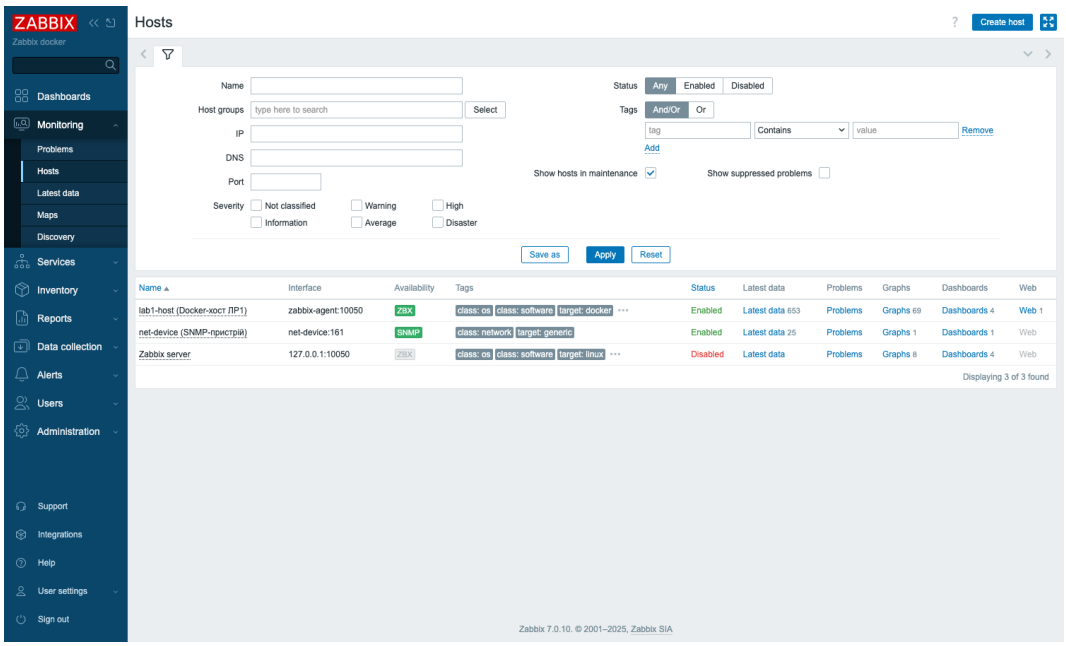


Рис. 3. Monitoring → Hosts: індикатори ZBX та SNMP активні

Пункт 4. Метрики шаблону Linux by Zabbix agent

Команда:

```
Monitoring → Latest data (item.get)
```

Результат:

```
$ .venv/bin/python zbx_query.py latest 'system.cpu.util' 'vm.memory.utilization' 'vfs.fs.depend
  ent.size[/etc/hosts' 'net.if.in[' | grep -E 'system.cpu.util |vm.memory.utilization|pused|net
  .if.in' | head -12
lab1-host  system.cpu.util                                    delay=0    last='3.841641999
  999993' % state=0
lab1-host  vfs.fs.dependent.size[/etc/hosts,pused]            delay=0    last='11.25214544
  6476135' % state=0
lab1-host  net.if.in["erspan0"]                               delay=3m   last='0' bps stat
```

```
  e=0
lab1-host  net.if.in["erspan0",dropped]                          delay=3m   last='0'  state=0
lab1-host  net.if.in["erspan0",errors]                           delay=3m   last='0'  state=0
net-device net.if.in[ifInOctets.11]                              delay=3m   last='176' bps st
  ate=0
lab1-host  net.if.in["eth0"]                                     delay=3m   last='7096' bps s
  tate=0
lab1-host  net.if.in["eth0",dropped]                             delay=3m   last='0'  state=0
lab1-host  net.if.in["eth0",errors]                              delay=3m   last='0'  state=0
lab1-host  net.if.in["gre0"]                                     delay=3m   last='0' bps stat
  e=0
lab1-host  net.if.in["gre0",dropped]                             delay=3m   last='0'  state=0
... (скорочено, усього 21 рядків)
```

Інтерпретація: Надходять метрики CPU (system.cpu.util), пам'яті (vm.memory.utilization), файлової системи (vfs.fs.dependent.size) та мережевого трафіку (net.if.in).

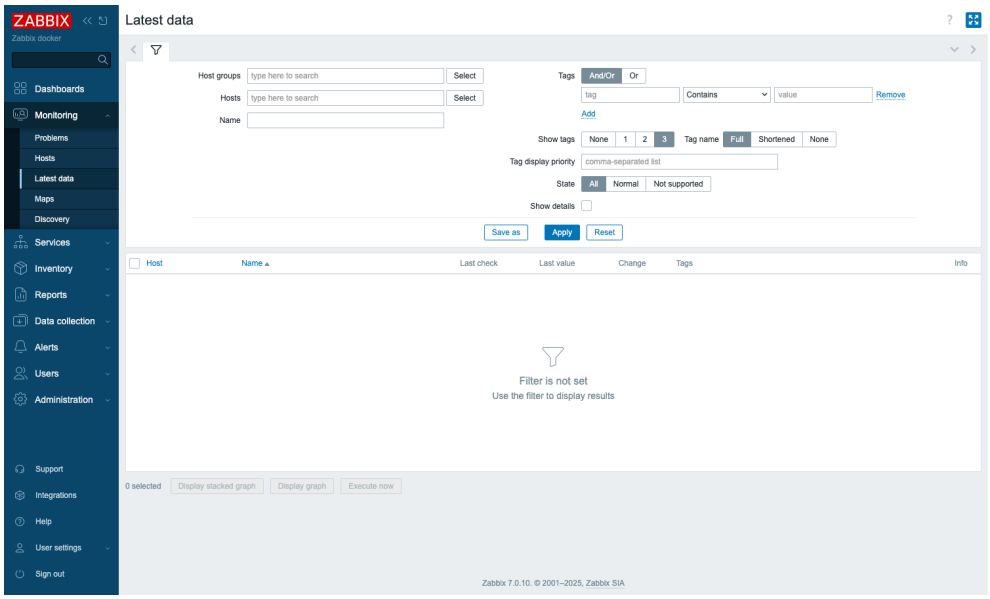


Рис. 4. Latest data: завантаження процесора

Приклад метрики: елемент «CPU utilization», ключ system.cpu.util, інтервал збору за шаблоном — залежний елемент від 1-хвилинних даних, останнє значення видно на рис. 4.

Пункт 5. Docker-плагін: стан контейнера БД

Команда:

```
Latest data для db
```

Результат:

```
$ .venv/bin/python zbx_query.py latest 'lab1.db' 'docker.container_info[db' 'docker.container_i
  nfo.state.status["/db"]'
lab1-host  docker.container_info.state.status["/db"]             delay=0    last='running'  s
  tate=0
lab1-host  docker.container_info[db,full]                        delay=10s  last='{"Id":"f1a5
  660c9a27b3614c4026a742c06f7e8'  state=0
lab1-host  lab1.db.up                                            delay=0    last='1'  state=0
lab1-host  lab1.db.state                                         delay=0    last='running'  s
  tate=0
```

Інтерпретація: Елемент docker.container_info.state.status["/db"] (шаблон Docker by Zabbix agent 2) і власний lab1.db.state мають значення running; lab1.db.up=1.

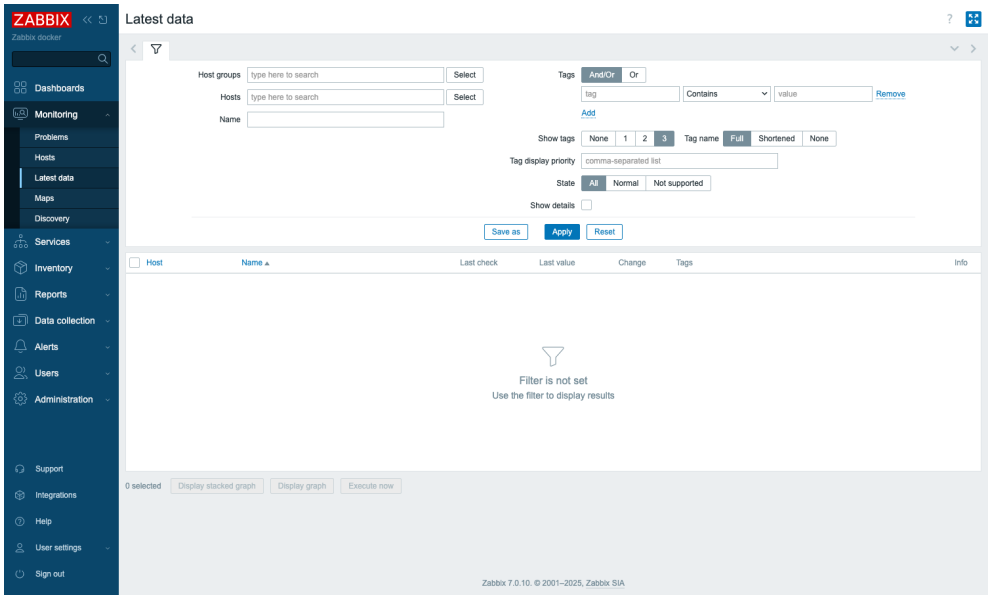


Рис. 5. Latest data: стан контейнерів ЛР1

Пункт 6. Веб-сценарій (безагентна перевірка)

Команда:

```
httptest lab1-app, крок proxy: http://proxy:80/, інтервал 10 с
```

Результат:

```
$ .venv/bin/python zbx_query.py latest 'web.test'
lab1-host  web.test.in[lab1-app,,bps]                          delay=10s  last='5201' Bps s
  tate=0
lab1-host  web.test.in[lab1-app,proxy,bps]                     delay=10s  last='5201' Bps s
  tate=0
lab1-host  web.test.fail[lab1-app]                             delay=10s  last='0'  state=0
lab1-host  web.test.error[lab1-app]                            delay=10s  last='response co
  de "503" did not match any of'  state=0
lab1-host  web.test.rspcode[lab1-app,proxy]                    delay=10s  last='200'  state
  =0
lab1-host  web.test.time[lab1-app,proxy,resp]                  delay=10s  last='0.007306' s
   state=0
```

Інтерпретація: Сценарій створив елементи web.test.rspcode (код відповіді, 200), web.test.time (час відповіді), web.test.in, web.test.fail та web.test.error; значення оновлюються кожні 10 с.

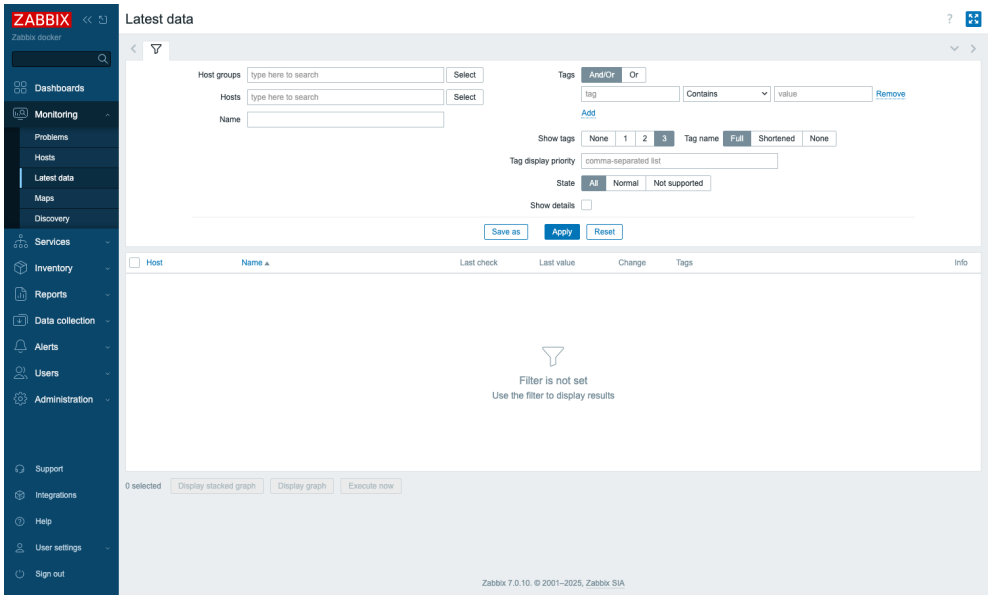


Рис. 6. Latest data: елементи веб-сценарію

Пункт 7. SNMP-пристрій: перевірка net-snmp

Команда:

```
snmpget/snmpwalk sysName, sysUpTime, ifDescr
```

Результат:

```
$ N='docker run --rm --network lab3-monitoring lab3-snmpd:1.0'; $N snmpget -v2c -c lab3comm net
  -device 1.3.6.1.2.1.1.5.0 1.3.6.1.2.1.1.3.0; $N snmpwalk -v2c -c lab3comm net-device 1.3.6.1.
  2.1.1; $N snmpwalk -v2c -c lab3comm net-device 1.3.6.1.2.1.2.2.1.2
SNMPv2-MIB::sysName.0 = STRING: net-device-1
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (71626) 0:11:56.26
SNMPv2-MIB::sysDescr.0 = STRING: Linux net-device 6.12.54-linuxkit #1 SMP Tue Nov  4 21:21:47 U
  TC 2025 aarch64
SNMPv2-MIB::sysObjectID.0 = OID: NET-SNMP-MIB::netSnmpAgentOIDs.10
DISMAN-EVENT-MIB::sysUpTimeInstance = Timeticks: (71708) 0:11:57.08
SNMPv2-MIB::sysContact.0 = STRING: student@example.com
SNMPv2-MIB::sysName.0 = STRING: net-device-1
SNMPv2-MIB::sysLocation.0 = STRING: Lab3 stand
SNMPv2-MIB::sysServices.0 = INTEGER: 72
SNMPv2-MIB::sysORLastChange.0 = Timeticks: (0) 0:00:00.00
SNMPv2-MIB::sysORID.1 = OID: SNMP-FRAMEWORK-MIB::snmpFrameworkMIBCompliance
SNMPv2-MIB::sysORID.2 = OID: SNMP-MPD-MIB::snmpMPDCompliance
SNMPv2-MIB::sysORID.3 = OID: SNMP-USER-BASED-SM-MIB::usmMIBCompliance
SNMPv2-MIB::sysORID.4 = OID: SNMPv2-MIB::snmpMIB
... (скорочено, усього 57 рядків)
```

Інтерпретація: OID 1.3.6.1.2.1.1.5.0 — sysName, 1.3.6.1.2.1.1.3.0 — sysUpTime, 1.3.6.1.2.1.1 — system MIB, 1.3.6.1.2.1.2.2.1.2 — ifDescr (перелік інтерфейсів; eth0 має індекс 11).

Обмеження доступу SNMP

Команда:

```
snmpget з невірною community; snmpd.conf
```

Результат:

```
$ docker run --rm --network lab3-monitoring lab3-snmpd:1.0 snmpget -v2c -c wrongcomm -t1 -r1 ne
  t-device 1.3.6.1.2.1.1.5.0; echo 'rocommunity ... 172.30.0.0/24 -> лише мережа моніторингу';
  grep -v '^#' snmpd/snmpd.conf
Timeout: No Response from net-device.
rocommunity ... 172.30.0.0/24 -> лише мережа моніторингу
agentAddress udp:0.0.0.0:161
rocommunity lab3comm 172.30.0.0/24
rocommunity lab3comm 127.0.0.1
sysName net-device-1
sysLocation Lab3 stand
sysContact student@example.com
sysServices 72
```

Інтерпретація: З неправильною community відповіді немає (тайм-аут), а rocommunity обмежено підмережею 172.30.0.0/24.

Пункт 8. SNMP-метрики в Zabbix

Команда:

```
Latest data net-device
```

Результат:

```
$ .venv/bin/python zbx_query.py latest 'snmp.' | grep net-device
net-device snmp.ifoper.eth0                                          delay=10s  last='1'  state=0
net-device snmp.uptime                                               delay=10s  last='715.51' s s
  tate=0
net-device snmp.ifout.eth0                                           delay=10s  last='144.6761408
  7500706' bps state=0
net-device snmp.ifin.eth0                                            delay=10s  last='132.6864054
  4337666' bps state=0
```

Інтерпретація: Три метрики отримано по SNMP: час роботи (OID ...1.3.0), стан інтерфейсу ifOperStatus (...2.2.1.8.11) і трафік eth0 (ifHCInOctets/ifHCOutOctets).

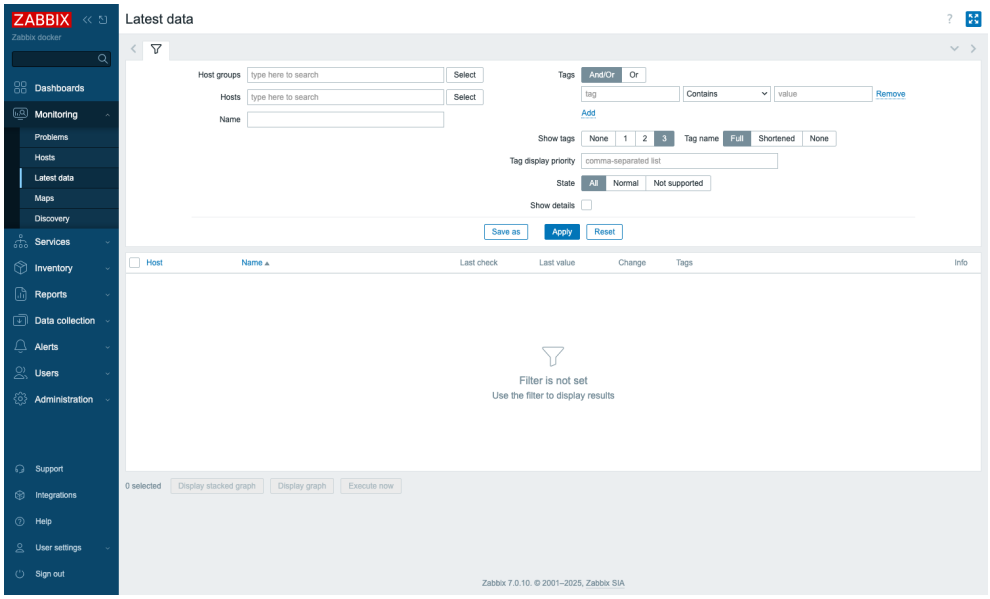


Рис. 7. Latest data: метрики SNMP-пристрою

Пункт 9. Тригери

Вузол	Тригер	Вираз	Важливість	Обґрунтування
lab1-host	Застосунок недоступний через проксі	last(/lab1-host/web.test.rspcode[lab1-app,proxy])<>200 or last(/lab1-host/web.test.fail[lab1-app])>0	High	Код не 200 означає, що сервіс не обслуговує запити; один збій достатньо, бо перевірка раз на 10 с
lab1-host	Контейнер БД зупинено або відсутній	last(/lab1-host/lab1.db.up)=0 or nodata(/lab1-host/lab1.db.up,60s)=1	High	БД критична для застосунку; nodata 60 с = 6 пропущених інтервалів (контейнер видалено або агент недоступний)
net-device	SNMP-пристрій недоступний	nodata(/net-device/snmp.uptime,60s)=1	Average	Немає SNMP-відповіді 60 с (6 інтервалів) — вважаємо пристрій недоступним
net-device	Вхідний трафік eth0 перевищує поріг	avg(/net-device/snmp.ifin.eth0,1m)>{$SNMP_IN_BPS_MAX}	Warning	Середнє за 1 хв згладжує піки; поріг 10 Мбіт/с задано макросом на вузлі

Налаштовані тригери

Команда:

```
trigger.get
```

Результат:

```
$ .venv/bin/python zbx_query.py triggers | grep -E 'LR1:|SNMP:'
net-device | SNMP: пристрій net-device недоступний | nodata(/net-device/snmp.uptime,60s)=1 | pr
  io 3 | value 0
net-device | SNMP: вхідний трафік eth0 перевищує поріг | avg(/net-device/snmp.ifin.eth0,1m)>100
  00000 | prio 2 | value 0
lab1-host | LR1: застосунок недоступний через проксі (HTTP != 200) | last(/lab1-host/web.test.r
  spcode[lab1-app,proxy])<>200 or last(/lab1-host/web.test.fail[lab1-app])>0 | prio 4 | value 0

lab1-host | LR1: контейнер БД зупинено або відсутній | last(/lab1-host/lab1.db.up)=0 or nodata(
```

```
  /lab1-host/lab1.db.up,60s)=1 | prio 4 | value 0
```

Інтерпретація: Усі чотири тригери перебувають у стані OK (value 0) до початку експерименту.

Пункт 10. Експеримент зі збоєм

Керований збій БД

Команда:

```
python fault_experiment.py (docker compose -p lab1 stop db / start db)
```

Результат:

```
17:49:59 зупинено db
17:50:02 з'явилась проблема: LR1: застосунок недоступний через проксі (HTTP != 200) (час події
  17:50:01)
17:50:11 з'явилась проблема: LR1: контейнер БД зупинено або відсутній (час події 17:50:09)
17:50:38 запущено db
17:50:41 проблему закрито: LR1: контейнер БД зупинено або відсутній (час закриття 17:50:39)
17:50:43 проблему закрито: LR1: застосунок недоступний через проксі (HTTP != 200) (час закриття
   17:50:41)

проблема | з'явилась | закрита | затримка виявлення, с | затримка закриття, с
LR1: застосунок недоступний через проксі (HTTP != 200) | 17:50:01 | 17:50:41 | 1.4 | 3.0
LR1: контейнер БД зупинено або відсутній | 17:50:09 | 17:50:39 | 9.4 | 1.0
зупинка db: 17:49:59, старт db: 17:50:38
```

Інтерпретація: Скрипт зупинив БД, дочекався проблем, відновив БД і дочекався автоматичного закриття проблем.

Подія	**Час**	**Затримка**
Зупинка db	17:49:59	-
LR1: застосунок недоступний через проксі (HTTP != 200) — виникла	17:50:01	1.4 с
LR1: контейнер БД зупинено або відсутній — виникла	17:50:09	9.4 с
Запуск db	17:50:38	-
LR1: контейнер БД зупинено або відсутній — закрита	17:50:39	1.0 с після старту
LR1: застосунок недоступний через проксі (HTTP != 200) — закрита	17:50:41	3.0 с після старту

Затримка виявлення не є сталою: вона залежить від моменту збою в циклі опитування. HTTP-проблему виявлено за ~1,4 с (збій припав на початок циклу веб-сценарію з інтервалом 10 с), а проблему БД за ~9,4 с — майже повний інтервал 10 с елемента lab1.db.up. Обидві затримки не перевищують інтервал опитування плюс час обчислення тригера, тож додаткових пояснень не потребують. Закриття проблем відбувається автоматично за наступним успішним опитуванням (затримка 1-3 с).

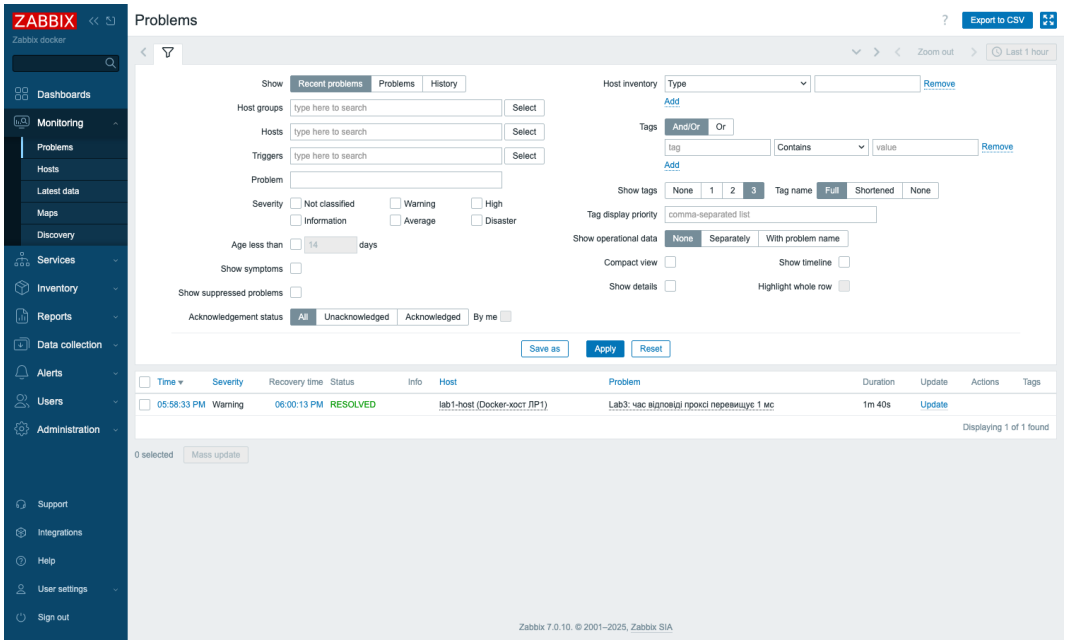


Рис. 8. Monitoring → Problems: проблеми виникли та закрито автоматично (Recovery time)

Пункт 11. Grafana

Grafana розгорнуто в мережі моніторингу з плагіном alexanderzobnin-zabbix-app 6.4.1. Datasource підключено до API Zabbix окремим користувачем grafana, який входить у групу grafana-readonly з правом лише читання (permission 2) на групу вузлів Lab3. Перевірено: користувач бачить обидва вузли, а спроба host.update завершується помилкою «No permissions to call host.update».



Рис. 9. Дашборд Grafana з періодом збою (db зупинено, HTTP-код 5xx)

- Доступність застосунку — код відповіді веб-сценарію у вигляді state-timeline; червоний інтервал відповідає збою БД (503);
- Час відповіді застосунку — відповідь кроку веб-сценарію в секундах;
- Стан контейнерів ЛР1 — запущено/зупинено (1/0) для db, proxy та app; червоний відрізок db збігається з експериментом;

• Трафік eth0 SNMP-пристрою — вхідний і вихідний трафік у біт/с, отриманий по SNMP.

Дашборд експортовано у grafana/dashboards/lab3.json, діапазон часу охоплює 2 хв до збою і 2 хв після відновлення.

2.5. Таблиця налаштованих елементів даних

Вузол	Тип	Ключ	Інтервал
lab1-host	Zabbix agent	docker.container_info[db,full] (+ залежні lab1.db.state, lab1.db.up)	10 с
lab1-host	Web (httptest)	web.test.rspcode[lab1-app,proxy], web.test.time[lab1-app,proxy,resp], web.test.fail[lab1-app]	10 с
lab1-host	Simple check	net.tcp.service.perf[http,proxy,80] (власний шаблон)	10 с
lab1-host	Calculated	lab3.rt.ms (власний шаблон)	10 с
net-device	SNMP agent	snmp.uptime (1.3.6.1.2.1.1.3.0)	10 с
net-device	SNMP agent	snmp.ifoper.eth0 (1.3.6.1.2.1.2.2.1.8.11)	10 с
net-device	SNMP agent	snmp.ifin.eth0 / snmp.ifout.eth0 (ifHCInOctets/OutOctets .11)	10 с

2.6. Додаткове завдання (високий рівень)

Власний шаблон «Lab3 HTTP template» містить елемент net.tcp.service.perf[http,proxy,80], обчислюваний елемент lab3.rt.ms (last(//net.tcp.service.perf[http,proxy,80])*1000), макрос {$LAB3.RT.MAX.MS}=500 та тригер avg(/Lab3 HTTP template/lab3.rt.ms,1m)>{$LAB3.RT.MAX.MS}. Шаблон застосовано до lab1-host.

Перевизначення макросу на рівні вузла

Команда:

```
python configure_template.py macro 1; потім macro reset
```

Результат:

```
$ ТРИГЕР ШАБЛОНУ ДО ПЕРЕВИЗНАЧЕННЯ МАКРОСУ ({$LAB3.RT.MAX.MS}=500 з шаблону)
lab1-host | Lab3: час відповіді проксі перевищує {$LAB3.RT.MAX.MS} мс | avg(/lab1-host/lab3.rt.
  ms,1m)>500 | prio 2 | value 0

$ python configure_template.py macro 1   # перевизначення на вузлі lab1-host
Макрос на рівні вузла перевизначено: 1
$ ПІСЛЯ ПЕРЕВИЗНАЧЕННЯ (через ~100 с)
lab1-host | Lab3: час відповіді проксі перевищує {$LAB3.RT.MAX.MS} мс | avg(/lab1-host/lab3.rt.
  ms,1m)>1 | prio 2 | value 1
17:58:33 Lab3: час відповіді проксі перевищує 1 мс severity 2 ACTIVE

$ python configure_template.py macro reset
Макрос на рівні вузла видалено (діє значення шаблону 500)
$ ПІСЛЯ ПОВЕРНЕННЯ МАКРОСУ
lab1-host | Lab3: час відповіді проксі перевищує {$LAB3.RT.MAX.MS} мс | avg(/lab1-host/lab3.rt.
  ms,1m)>500 | prio 2 | value 0
17:58:33 Lab3: час відповіді проксі перевищує 1 мс severity 2 resolved
```

Інтерпретація: З макросом шаблону (500 мс) тригер у стані OK. Після перевизначення на вузлі до 1 мс вираз став avg(...)>1, тригер перейшов у PROBLEM; після скидання макросу повернувся 500 і проблему закрито. Отже, поведінка тригера змінюється без редагування шаблону.

Розширення (а) — low-level discovery. Обґрунтування вибору: LLD найкраще відповідає динаміці контейнерного середовища, де кількість реплік змінюється без участі адміністратора, а ручне додавання елементів і тригерів для кожної репліки

неприйнятне.

Автовиявлення контейнерів: нова репліка app

Команда:

```
docker compose -p lab1 up -d --scale app=2; Latest data lab3.container.up
```

Результат:

```
$ ДО МАСШТАБУВАННЯ
lab1-host  lab3.container.up[/db]                                    delay=0    last='1'  state=0
lab1-host  lab3.container.up[/lab1-app-1]                            delay=0    last='1'  state=0
lab1-host  lab3.container.up[/proxy]                                 delay=0    last='1'  state=0

$ docker compose -p lab1 up -d --scale app=2 --no-recreate
 Container lab1-app-1  Waiting
 Container lab1-app-2  Healthy
 Container lab1-app-1  Healthy

$ ПІСЛЯ МАСШТАБУВАННЯ (через 75 с, LLD кожні 30 с)
lab1-host  lab3.container.up[/db]                                    delay=0    last='1'  state=0
lab1-host  lab3.container.up[/lab1-app-1]                            delay=0    last='1'  state=0
lab1-host  lab3.container.up[/lab1-app-2]                            delay=0    last='1'  state=0
lab1-host  lab3.container.up[/proxy]                                 delay=0    last='1'  state=0
$ python zbx_query.py triggers | grep 'Lab3: контейнер'  (тригери, створені з прототипу)
lab1-host | Lab3: контейнер /proxy зупинено | last(/lab1-host/lab3.container.up[/proxy])=0 | pr
  io 3 | value 0
lab1-host | Lab3: контейнер /lab1-app-1 зупинено | last(/lab1-host/lab3.container.up[/lab1-app-
  1])=0 | prio 3 | value 0
lab1-host | Lab3: контейнер /db зупинено | last(/lab1-host/lab3.container.up[/db])=0 | prio 3 |
   value 0
lab1-host | Lab3: контейнер /lab1-app-2 зупинено | last(/lab1-host/lab3.container.up[/lab1-app-
  2])=0 | prio 3 | value 0
```

Інтерпретація: Правило виявлення docker.containers.discovery[true] (інтервал 30 с, фільтр за {#NAME}) створило прототипи: після масштабування з'явились елемент lab3.container.up[/lab1-app-2] і тригер «контейнер /lab1-app-2 зупинено» без ручного налаштування.

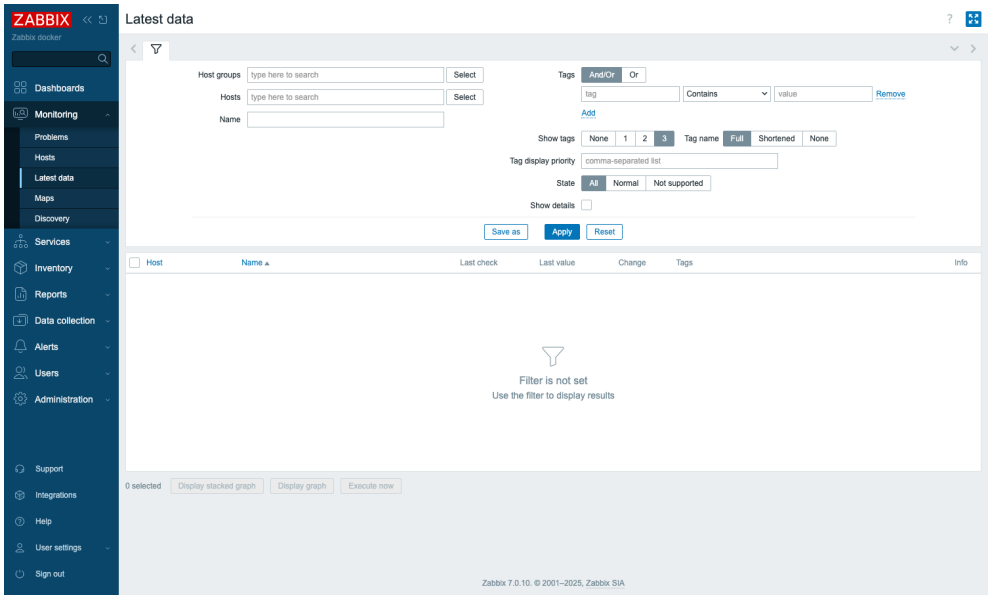


Рис. 10. Latest data: елементи, створені правилом виявлення, включно з реплікою lab1-app-2

3. Конфігураційні файли

```
name: lab3

services:
  zabbix-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: ${ZBX_DB_PASSWORD}
```

```
      POSTGRES_DB: zabbix
    volumes:
      - zabbix-db-data:/var/lib/postgresql/data
    networks: [monitoring]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U zabbix"]
      interval: 5s
      timeout: 3s
      retries: 20

  zabbix-server:
    image: zabbix/zabbix-server-pgsql:alpine-7.0.10
    environment:
      DB_SERVER_HOST: zabbix-db
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: ${ZBX_DB_PASSWORD}
      POSTGRES_DB: zabbix
    depends_on:
      zabbix-db:
        condition: service_healthy
    # monitoring: власна мережа; lab1-frontend: щоб веб-сценарій бачив proxy з ЛР1
    networks: [monitoring, lab1]

  zabbix-web:
    image: zabbix/zabbix-web-nginx-pgsql:alpine-7.0.10
    environment:
      ZBX_SERVER_HOST: zabbix-server
      DB_SERVER_HOST: zabbix-db
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: ${ZBX_DB_PASSWORD}
      POSTGRES_DB: zabbix
      PHP_TZ: Europe/Kyiv
    depends_on:
      - zabbix-server
    # єдиний порт веб-інтерфейсу Zabbix
    ports:
      - "8081:8080"
    networks: [monitoring]

  zabbix-agent:
    image: zabbix/zabbix-agent2:alpine-7.0.10
    # root потрібен для читання docker.sock
    user: root
    hostname: lab1-host
    environment:
      ZBX_HOSTNAME: lab1-host
      ZBX_SERVER_HOST: zabbix-server
    volumes:
      # Docker Engine лише для читання
      - /var/run/docker.sock:/var/run/docker.sock:ro
    networks: [monitoring]

  net-device:
    build: ./snmpd
    image: lab3-snmpd:1.0
    hostname: net-device
    networks: [monitoring]

  grafana:
    image: grafana/grafana:11.6.0
    environment:
      GF_INSTALL_PLUGINS: alexanderzobnin-zabbix-app 6.4.1
      GF_PLUGINS_ALLOW_LOADING_UNSIGNED_PLUGINS: alexanderzobnin-zabbix-datasource
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD}
      GRAFANA_ZBX_PASSWORD: ${GRAFANA_ZBX_PASSWORD}
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
      - grafana-data:/var/lib/grafana
    ports:
      - "3000:3000"
    networks: [monitoring]
    depends_on:
      - zabbix-web

networks:
```

```
  monitoring:
    name: lab3-monitoring
    driver: bridge
    ipam:
      config:
        - subnet: 172.30.0.0/24
          gateway: 172.30.0.1
  lab1:
    # мережа проєкту ЛР1 (лише підключення, проєкт не змінюється)
    name: lab1-frontend
    external: true

volumes:
  zabbix-db-data:
  grafana-data:
```

docker-compose.yml

```
# слухати на всіх інтерфейсах контейнера
agentAddress udp:0.0.0.0:161
# SNMPv2c, доступ лише з мережі моніторингу
rocommunity lab3comm 172.30.0.0/24
# локальна діагностика всередині контейнера
rocommunity lab3comm 127.0.0.1
sysName net-device-1
sysLocation Lab3 stand
sysContact student@example.com
sysServices 72
```

snmpd/snmpd.conf

```
FROM alpine:3.20
# net-snmp: демон snmpd та утиліти snmpwalk/snmpget
RUN apk add --no-cache net-snmp net-snmp-tools
COPY snmpd.conf /etc/snmp/snmpd.conf
EXPOSE 161/udp
# -f: передній план, -Lo: лог у stdout, -C: не читати типові конфіги (інакше порт реєструється двічі)
CMD ["snmpd", "-f", "-Lo", "-C", "-c", "/etc/snmp/snmpd.conf"]
```

snmpd/Dockerfile

```
#!/usr/bin/env bash
# Розгортання системи моніторингу однією командою (потрібен запущений проєкт ЛР1)
set -euo pipefail
cd "$(dirname "$0")"
# паролі генеруються локально і не потрапляють у git
if [[ ! -f .env ]]; then
  {
    echo "ZBX_DB_PASSWORD=$(openssl rand -hex 12)"
    echo "ZBX_ADMIN_PASSWORD=$(openssl rand -base64 12 | tr -d '/+=')Aa1"
    echo "GRAFANA_ADMIN_PASSWORD=$(openssl rand -base64 12 | tr -d '/+=')Aa1"
    echo "GRAFANA_ZBX_PASSWORD=$(openssl rand -base64 12 | tr -d '/+=')Aa1"
  } > .env
  chmod 600 .env
fi
docker network inspect lab1-frontend >/dev/null 2>&1 || { echo "Спочатку запустіть проєкт ЛР1 (docker compos
docker compose up -d --build
[[ -x .venv/bin/python ]] || python3 -m venv .venv
.venv/bin/pip install -q -r requirements.txt
# налаштування Zabbix через API
.venv/bin/python configure_zabbix.py
```

up.sh

```
apiVersion: 1
datasources:
  - name: Zabbix
    uid: zabbix
    type: alexanderzobnin-zabbix-datasource
    access: proxy
    url: http://zabbix-web:8080/api_jsonrpc.php
    isDefault: true
    jsonData:
      # окремий користувач Zabbix лише для читання
      username: grafana
      trends: true
      trendsFrom: 7d
      cacheTTL: 30s
    secureJsonData:
      password: $__env{GRAFANA_ZBX_PASSWORD}
```

grafana/provisioning/datasources/zabbix.yml

```
zabbix_export:
  version: '7.0'
  template_groups:
    - uuid: 7df96b18c230490a9a0a9e2307226338
      name: Templates
  templates:
    - uuid: 915a2b9901e24fcc9689a291e146d40d
      template: 'Lab3 HTTP template'
      name: 'Lab3 HTTP template'
      groups:
        - name: Templates
      items:
        - uuid: a4508b56474f417db0e9f0ebebc53f43
          name: 'Lab3: час відповіді HTTP проксі, мс'
          type: CALCULATED
          key: lab3.rt.ms
          delay: 10s
          history: 7d
          value_type: FLOAT
          units: ms
          params: 'last(//net.tcp.service.perf[http,proxy,80])*1000'
          triggers:
            - uuid: 174c876459e4421d8533503d5cf77cd6
              expression: 'avg(/Lab3 HTTP template/lab3.rt.ms,1m)>{$LAB3.RT.MAX.MS}'
              name: 'Lab3: час відповіді проксі перевищує {$LAB3.RT.MAX.MS} мс'
              priority: WARNING
        - uuid: 5412349bfaa94c15a550b3f8b0e23a68
          name: 'Lab3: час відповіді HTTP проксі, с'
          type: SIMPLE
          key: 'net.tcp.service.perf[http,proxy,80]'
          delay: 10s
          history: 7d
          value_type: FLOAT
          units: s
      macros:
        - macro: '{$LAB3.RT.MAX.MS}'
          value: '500'
          description: 'Поріг часу відповіді проксі, мс'
```

export/lab3_http_template.yaml

4. Висновки

Розгорнуто систему моніторингу Zabbix 7.0.10 з Grafana в контейнерах однією командою up.sh; моніторинг зовнішній щодо проєкту ЛР1. Налаштовано три способи збору даних, чотири тригери та підтверджено автоматичне виявлення і закриття збоїв: затримка виявлення не перевищила інтервал опитування. Власний шаблон із макросом дозволив змінювати поріг на рівні вузла, а low-level discovery автоматично охопив нову репліку застосунку.

Порівняння способів збору даних:

Критерій	**Zabbix agent**	**SNMP**	**Веб-сценарій**
Що потрібно на цільовому вузлі	Встановлений агент і доступ до даних (наприклад, docker.sock)	SNMP-агент (snmpd) і налаштована community/ACL	Нічого: достатньо доступного URL
Хто ініціює обмін	Сервер (пасивні) або агент (активні перевірки)	Сервер опитує пристрій (polling); можливі trap	Сервер виконує HTTP-запити
Затримка виявлення збою	До інтервалу опитування + обчислення тригера (тут ~9 с при 10 с)	До інтервалу опитування + тайм-аути SNMP	До інтервалу сценарію (тут 10 с; виявлено за 1,4 с)
Придатність для мережевого обладнання	Низька: на комутатор агент зазвичай не встановити	Висока: SNMP є майже на всьому обладнанні	Лише для перевірки доступності сервісів, не пристрою

Висновок: агент доцільний для серверів і контейнерних хостів (глибокі метрики), SNMP — для мережевого обладнання, веб-сценарій — для оцінки доступності сервісу очима користувача. Найкраще їх поєднувати.

Підпис виконавця: _____ Підпис викладача: _____